Data Structure, Quiz 1 (2026)

A. Single Choice Questions (5% each, 60%)

(1) Which of the following statements about abstract data types, data structures, and classes is TRUE?

(A) An ADT specifies exactly how its data must be laid out in memory.

(B) Encapsulation means that a user must read the implementation of an ADT before using its operations.

(C) A class describes what the data look like (the state) and what the data can do (the behavior); a user of a class sees only this state and behavior, which is why classes are analogous to ADTs.

(D) The data structure is the logical view of the data, while the ADT is its physical implementation.

(2) In the logic-gate simulation from class, `LogicGate` is the parent class; `BinaryGate` and `UnaryGate` are derived from it, and `AndGate` is derived from `BinaryGate`. `LogicGate` declares the operation `performGateLogic` but does not implement it; each concrete gate implements it. A `Connector` object holds two gates, one at each end. Which statement is FALSE?

(A) An `AndGate` **Is-a** `LogicGate`.

(B) A `Connector` **Has-a** `LogicGate`; it is not part of the gate hierarchy.

(C) `LogicGate` is an abstract class, so no plain `LogicGate` object can be created; each concrete gate overrides `performGateLogic`.

(D) Because a `Connector` uses gates, `Connector` must be a subclass of `LogicGate`.

(3) Two algorithms solve the same problem. Their exact step counts are

$$T_P(n) = 100n + 5000, \qquad T_Q(n) = \frac{n^2}{2}.$$

Which statement is TRUE?

(A) P is $O(n)$ and Q is $O(n^2)$. Q needs fewer steps for small inputs such as $n = 100$, but P needs fewer steps once n is large enough.

(B) Because T_P has much larger constants, Q is faster than P for every input size.

(C) Both have the same dominant term n^2, because the constant 5000 is larger than n for small inputs.

(D) P is $O(100n)$, which is a different Big-O class from $O(n)$.

(4) Match the four anagram-detection solutions from class with their order of magnitude, in this order: (1) Checking Off, (2) Sort and Compare, (3) Brute Force, (4) Count and Compare. For Brute Force, count how many candidate strings it must generate; assume sorting costs $O(n \log n)$.

(A) $O(n)$, $O(n \log n)$, $O(2^n)$, $O(n)$

(B) $O(n^2)$, $O(n \log n)$, $O(n!)$, $O(n)$

(C) $O(n^2)$, $O(n)$, $O(n!)$, $O(n)$

(D) $O(n^2)$, $O(n \log n)$, $O(n!)$, $O(1)$

(5) Four ways to build a dynamic array holding n integers were compared in class:

1. insert each new element at the **front** of the array;
2. append each new element at the **end**;
3. reserve capacity for n elements first, then append each element at the end;
4. create the array with n slots first, then assign each element by index.

Which statement is TRUE?

(A) Every single append in Method 2 takes $O(1)$ time in the worst case.

(B) Method 3 has a better Big-O than Method 2, because reserving capacity avoids reallocations.

(C) Method 1 takes $O(n^2)$ time in total, because every insertion at the front shifts all existing elements.

(D) Method 4 takes $O(n^2)$ time, because creating n slots already costs $O(n)$ for each slot.

(6) Assume an integer uses 4 bytes and a memory reference (address) uses 8 bytes. A **compact array** stores 10 integers directly. A **referential array** stores 10 references, each referring to its own integer stored elsewhere in memory. Counting the integers themselves, how many bytes does each layout use in total?

(A) 40 bytes and 120 bytes

(B) 40 bytes and 80 bytes

(C) 40 bytes and 40 bytes

(D) 80 bytes and 120 bytes

(7) Which of the following statements about storing a sparse matrix is TRUE?

(A) A matrix is called sparse when more than half of its entries are non-zero.

(B) The linear-list format stores the non-zero entries in an array sorted by their values.

(C) The Dictionary of Keys (DOK) format stores every entry of the matrix, including the zeros, as key-value pairs.

(D) The COOrdinate (COO) format stores each non-zero entry using three arrays: one for row indices, one for column indices, and one for values.

(8) Two empty lists are created with the List ADT: `u = UnorderedList()` (whose `add` inserts each new item the way the unordered list from class does) and `o = OrderedList()` (ascending order). Both lists then receive the same operations in this order:

```
add(5)   add(12)   add(3)   add(9)
```

What are the contents of each list from the head to the last node?

(A) `u` : 5 12 3 9; `o` : 3 5 9 12

(B) `u` : 9 3 12 5; `o` : 12 9 5 3

(C) `u` : 9 3 12 5; `o` : 3 5 9 12

(D) `u` : 3 5 9 12; `o` : 3 5 9 12

(9) The `remove(item)` operation of an unordered linked list traverses the list with two references: `current` marks the node being examined and `previous` follows one node behind it. The list has **at least two nodes**, and the item is found in the **first** node, so `previous` does not refer to any node. What must be changed to remove that node correctly?

(A) `head` must be changed to refer to the node after `current` , i.e. the second node.

(B) The link of `previous` must be changed to refer to the node after `current` .

(C) `head` must be set to refer to no node, because the list becomes empty.

(D) Nothing needs to change; the node will simply be skipped during later traversals.

(10) Which of the following statements about linked lists is FALSE?

(A) The nodes of a linked list do not have to be stored in contiguous memory; each node keeps a link to the next node.

(B) Accessing the item at position i of a singly linked list takes $O(1)$ time, just like indexing an array, because the head is known.

(C) In a doubly linked list each node keeps links to both the previous and the next node, which allows $O(1)$ insertions and deletions at arbitrary positions once the node is known.

(D) Adding an item at the head of an unordered linked list takes $O(1)$ time.

(11) A stack is implemented with a dynamic array in two different ways:

- **Version 1** keeps the top of the stack at the **end** of the array.
- **Version 2** keeps the top of the stack at the **beginning** (index 0) of the array.

Starting from an empty stack, a program pushes n items and then pops all n items. Which statement is TRUE?

(A) Both versions take $O(n)$ time in total, because both obey the LIFO rule.

(B) Both versions take $O(n^2)$ time in total, because a dynamic array shifts its elements on every operation.

(C) Version 1 takes $O(n^2)$ time in total, while Version 2 takes $O(n)$ time in total.

(D) Version 1 takes $O(n)$ time in total, while Version 2 takes $O(n^2)$ time in total.

(12) The values 1, 2, 3, 4 are pushed onto an initially empty stack **in this order**. A `pop` may be performed at any time (between pushes or after them), and each popped value is printed. Which of the following output sequences is **impossible**?

(A) `1 2 3 4`

(B) `3 1 4 2`

(C) `2 4 3 1`

(D) `4 3 2 1`

B. Short-answer questions, please provide the derivation for each question along with your answer (8% each, 40%)

(13) Briefly explain each of the following terms.

(a) Abstract data type (ADT)

(b) Column-major order

(c) Ordered list

(d) FIFO (first in, first out)

(14-1) Evaluate the time complexity of the following code segment in terms of Big-O notation (assume no integer overflow):

```
int x = 0;
for (int i = 1; i <= n; i *= 2) {
    for (int j = 0; j < n; j++) {
        x++;
    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < 100; j++) {
        x--;
    }
}

for (int k = 1; k < n * n; k *= 3) {
    x++;
}
```

(14-2) Evaluate the time complexity of the following code segment in terms of Big-O notation (assume no integer overflow):

```
x = 0
i = 1
while i <= n:
    for j in range(n):
        x += 1
    i *= 2

for i in range(n):
    for j in range(100):
        x -= 1

k = 1
while k < n * n:
    x += 1
    k *= 3
```

(15) Consider the conversion of the infix expression

`A * ( B + C * D ) - E / F`
to postfix form using a stack, as described in class.

(a) What is the resulting postfix expression?

(b) What is the maximum number of symbols (operators and left parentheses) in the operator stack at any one moment during the conversion?

(c) What is the equivalent prefix expression?

(16) Assume `q` is a queue that supports the Queue ADT operations (`enqueue`, `dequeue` returns the removed item, `isEmpty`, `size`) and `s` is a stack that supports the Stack ADT operations (`push`, `pop` returns the removed item, `peek`, `isEmpty`, `size`). Trace the following operations:

```
q = Queue()
s = Stack()
q.enqueue(4)
q.enqueue(9)
q.enqueue(2)
s.push(q.dequeue())
q.enqueue(s.peek() + 1)
s.push(q.dequeue())
s.push(7)
q.enqueue(s.pop())
```

(a) What are the final contents of `q` from front to rear?

(b) What are the final contents of `s` from bottom to top, and what does `s.peek()` return now?

(c) Which values were returned by the two `dequeue()` calls and by the `pop()` call?

(17) A two-dimensional array `X` is stored contiguously in memory on a computer whose storage order (row-major or column-major) is not known in advance (zero-based indices; every index mentioned below is valid). It is known that

- `X[1][2]` is located at address **244**
- `X[3][0]` is located at address **212**
- `X[4][3]` is located at address **276**

(a) Is the storage order row-major or column-major? Justify your answer.

(b) What is the size of each element?

(c) How many rows does `X` have?

(d) What is the address of `X[0][0]`?

(e) What is the address of `X[2][4]`?